Architecture des Ordinateurs : Représentation des Entiers Signés et Arithmétique en Complément à Deux

Thème principal : Représentation bas niveau des données et arithmétique binaire au sein de l'Unité Arithmétique et Logique (ALU)

Synthèse exécutive : Ce cours détaille le fonctionnement physique et logique du codage des entiers relatifs dans les ordinateurs. Face à l'impossibilité d'enregistrer des signes '+' ou '-' sur des composants électroniques bistables (qui ne stockent que des 0 ou des 1), l'architecture matérielle utilise la convention du complément à deux sur N bits. Cette représentation élimine le problème du double zéro présent dans la méthode 'Signe et Magnitude', et permet à l'Unité Arithmétique et Logique (ALU) d'utiliser le même circuit d'addition pour effectuer à la fois des additions et des soustractions (A - B = A + (-B)). Le cours explicite la méthode rigoureuse de conversion, le calcul des plages de valeurs dissymétriques, ainsi que la différenciation cruciale entre la retenue sortante (Carry out) et le dépassement de capacité arithmétique (Overflow).

1. Problématique matérielle et justification de la convention du Complément à Deux

Au niveau matériel, les registres d'un processeur sont composés de circuits électroniques bistables. Un ordinateur ne 'comprend' pas les symboles abstraits comme le signe moins. Pour représenter les nombres négatifs, plusieurs conventions ont été historiquement testées. La méthode la plus intuitive, 'Signe et Magnitude', pose deux problèmes majeurs : elle introduit deux expressions différentes pour la valeur zéro, ce qui complexifie les opérations d'égalité en imposant de tester deux motifs binaires distincts ; de plus, elle nécessite d'implanter deux circuits physiques distincts dans l'ALU (un pour additionner, un pour soustraire). Le complément à deux résout l'ensemble de ces limitations de manière élégante : le zéro possède une signature binaire unique (tous les bits à 0) et la soustraction est réintroduite sous forme d'une addition avec l'opposé du nombre.

- Absence de signe physique : Les bascules et transistors de la mémoire ne stockent que deux états logiques (0 pour 0 Volt, 1 pour Vcc). Il n'existe aucun composant physique dédié au symbole '+' ou '-'.
- Échec de la représentation Signe + Magnitude : Cette méthode réserve le bit le plus à gauche pour le signe (0 = positif, 1 = négatif) et le reste pour la valeur absolue. Elle produit deux représentations pour zéro (+0 = 0000_2 et -0 = 1000_2 sur 4 bits), ce qui gaspille une combinaison et exige de fabriquer des circuits d'addition et de soustraction séparés.
- Unification de l'ALU par le Complément à Deux : Cette convention attribue au zéro une représentation unique (0000...0) et permet de transformer toute soustraction A - B en une addition A + (-B), exécutée directement par le circuit additionneur de l'ALU.

Cas concret enseignant : *La métaphore du compteur kilométrique d'une voiture ancienne à 4 chiffres : si le compteur est bloqué à 0000 km et que l'on effectue un kilomètre en marche arrière, le compteur repasse mécaniquement à 9999. En binaire sur 8 bits, soustraire 1 à 00000000_2 donne 11111111_2, ce qui constitue la définition exacte de -1 en complément à deux.*

2. Algorithme de Conversion et Plage de Valeurs sur N bits

L'application de la méthode de conversion exige de travailler sur un mot mémoire de taille fixe prédéfinie (N bits). Il est impératif de formater d'abord la valeur absolue positive en complétant le champ avec des zéros à gauche jusqu'à atteindre exactement N bits. Une fois ce cadrage effectué, le complément à un consiste à inverser chaque bit de la séquence (0 devient 1, 1 devient 0). Enfin, l'ajout de la valeur 1 produit la représentation exacte en complément à deux. En raison de l'inclusion du nombre zéro parmi les représentations commençant par un bit de signe à 0, la plage des nombres représentables est dissymétrique : le domaine négatif contient un nombre de plus que le domaine positif.

• Algorithme des 3 étapes : Pour coder un nombre négatif -X, il faut : 1. Écrire X en binaire sur N bits (en ajoutant des 0 à gauche) ; 2. Inverser chaque bit (Complément à 1) ; 3. Ajouter 1 au résultat (Complément à 2).

• Bit de poids fort (MSB - Most Significant Bit) : Le bit le plus à gauche indique le signe (0 = positif ou nul, 1 = négatif).

• Plage de valeurs dissymétrique : Sur N bits, la plage s'étend de -2^(N-1) à +(2^(N-1) - 1). Sur 8 bits, cela correspond à [-128, +127].

Cas concret enseignant : *Avertissement crucial pour les contrôles et partiels : il faut IMPÉRATIVEMENT fixer la taille N bits (ex: 8 bits) AVANT d'effectuer l'inversion des bits. Inverser la chaîne '1100' (4 bits) sans ajouter au préalable les zéros de gauche '00001100' produit un résultat complètement faux !*

3. Arithmétique, Retenue (Carry Out) et Détection du Dépassement de Capacité (Overflow)

Dans un registre de taille fixe, toute opération arithmétique peut produire deux indicateurs distincts au niveau du registre d'état (drapeaux/flags du processeur) : le Carry (C) et l'Overflow (V). En arithmétique signée en complément à deux, le drapeau Carry Out est ignoré dans l'interprétation du résultat car le processeur fonctionne selon une arithmétique modulaire de période 2^N. En revanche, le drapeau d'Overflow est fondamental : il signale une erreur d'amplitude où le bit MSB a été corrompu par le calcul arithmétique.

• Gestion de la Retenue Sortante (Carry Out) : Lors d'une addition en complément à deux, la retenue générée au-delà du bit de poids fort est systématiquement ignorée car les calculs s'effectuent modulo 2^N.

• Définition de l'Overflow (OV) : Le dépassement de capacité survient lorsque le résultat mathématique de l'opération sort de la plage autorisée [-2^(N-1), +2^(N-1)-1].

• Règle d'or de détection d'Overflow : Un Overflow se produit si et seulement si deux opérandes de même signe (deux positifs ou deux négatifs) sont additionnés et produisent un résultat de signe opposé. Si les deux opérandes ont des signes différents, l'Overflow est physiquement impossible.

Cas concret enseignant : *L'anecdote historique de l'accident du vol 501 d'Ariane 5 (1996) : une conversion d'un nombre flottant 64 bits vers un entier signé 16 bits sans mécanisme de protection contre le dépassement de capacité a provoqué un Overflow non géré. L'erreur d'interprétation des données par le calculateur de bord a causé l'autodestruction de la fusée quelques secondes après son décollage.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la retenue tout à gauche (Carry sortant) est-elle jetée lors de l'addition en complément à deux ?
Réponse du professeur : Le matériel fonctionne sur des registres de taille fixe N bits. L'arithmétique effectuée est une arithmétique modulaire modulo 2^N. La retenue sortante dépasse la capacité physique du registre de N bits et tombe dans le drapeau Carry de l'ALU. Dans le système du complément à deux, enlever 2^N revient à conserver exactement le même reste, ce qui garantit la justesse de la représentation signée.

Question étudiant : Comment l'ordinateur fait-il la différence entre le nombre binaire 11111111 qui vaut 255 en non-signé et -1 en complément à deux ?
Réponse du professeur : L'ordinateur et la mémoire ne font aucune différence : ils ne stockent que 8 bits '11111111' sans étiquette contextuelle. C'est le programmeur (via le type de variable déclaré en C/Java) et le jeu d'instructions assembleur utilisé (ex: instructions d'addition ou de comparaison signées vs non signées) qui interprètent ces bits soit comme 255, soit comme -1.

■ Exercice & Démonstration d'Application

Énoncé : Sur une architecture 8 bits, effectuer la soustraction signée : 35 - 75. Détailler l'écriture binaire des deux opérandes, poser et effectuer l'addition binaire correspondante, donner le résultat décimal associé et analyser la présence ou l'absence d'un Overflow.

Résolution pas à pas :
1. Codage de +35 sur 8 bits : 35 = 32 + 2 + 1 = 00100011_2. 2. Codage de -75 en complément à deux sur 8 bits : - Valeur absolue (+75) sur 8 bits : 64 + 8 + 2 + 1 = 01001011_2. - Inversion des bits (Complément à 1) : 10110100_2. - Ajout de 1 (Complément à 2) : 10110100 + 1 = 10110101_2 (Représentation de -75). 3. Pose de l'addition (+35) + (-75) : 00100011 (+35) + 10110101 (-75) ---------- 11011000_2 4. Interprétation du résultat : Le bit MSB vaut 1, le résultat est donc négatif. Pour retrouver sa valeur absolue, on réapplique le complément à deux sur 11011000_2 : - Inversion des bits : 00100111_2 - Ajout de 1 : 00101000_2 = 32 + 8 = 40 décimal. Le résultat binaire 11011000_2 correspond donc à -40 en décimal. 5. Analyse du dépassement de capacité (Overflow) : On a additionné un nombre positif (+35, MSB=0) et un nombre négatif (-75, MSB=1). Les deux opérandes étant de signes opposés, un Overflow est mathématiquement impossible.

Résultat : Résultat binaire : 11011000_2 (-40 en décimal). Drapeau d'Overflow = 0. Le calcul 35 - 75 = -40 est parfaitement juste.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la plage exacte des entiers relatifs représentables sur un mot de 16 bits en complément à deux ?

R : La plage s'étend de -2^(16-1) à +(2^(16-1) - 1), soit de -32 768 à +32 767.

Q : Quelle est la condition nécessaire et suffisante pour détecter un Overflow lors de l'addition de deux entiers signés ?

R : L'Overflow se produit si et seulement si on additionne deux opérandes de même signe et que le résultat présente un signe opposé.

Q : Quel est le résultat de la procédure du complément à deux appliquée au nombre zéro (00000000_2) sur 8 bits ?

R : L'inversion de 00000000 donne 11111111. L'ajout de 1 donne 100000000_2. Sur 8 bits, le bit de retenue tout à gauche est rejeté, il reste 00000000_2. Cela prouve l'unicité de la représentation du zéro.